

PARTE I

Bubble Sort

- **Como funciona:** O Bubble Sort é um algoritmo que compara e troca os pares vizinhos que estão ordenados de maneira errada. Ele percorre os números e os compara, se estiverem ordenados de maneira errada ele realiza a troca. Isso se repete várias vezes até não ter mais nenhuma troca pra fazer.
- **Melhor caso:** $O(n)$. É quando o array já está ordenado — com uma versão otimizada (usando uma flag), o algoritmo passa uma vez, vê que não teve troca nenhuma e já para.
- **Caso médio:** $O(n^2)$. É quando o array está em ordem aleatória, aí precisa de várias passadas com bastante comparação e troca.
- **Pior caso:** $O(n^2)$. É quando o array está totalmente ao contrário, tipo decrescente quando deveria estar crescente. Nesse caso o algoritmo faz o máximo de comparação e troca possível.
- **Vantagens:** É mais fácil de entender já que só vai trocando os elementos vizinhos, e também não gasta muita memória, porque ele troca dentro do próprio array, só usando uma variável temporária pra fazer a troca.
- **Limitações:** Ele demora mais pra realizar as trocas quando o array é grande, pois troca um elemento por vez, então se um número menor estiver muito distante do lugar dele, vai demorar muito mais pra concluir todo o processo.
- **Quando usar:** Poderíamos usar ele numa contagem regressiva que está desorganizada, por exemplo, já que são poucos elementos.
- **Quando não usar:** Pra comparar dados de uma empresa por exemplo, que são muitos dados, ele fica muito lento.

Quick Sort

- **Como funciona:** O Quick Sort é um algoritmo que usa um número como pivô pra organizar os números. A ideia é escolher um elemento como pivô, colocar os menores que ele à esquerda e os maiores à direita, e repetir esse processo de novo em cada metade. É muito mais eficiente que o Bubble Sort.
- **Melhor caso:** $O(n \log n)$. É quando o pivô sempre divide o vetor em partes praticamente iguais.
- **Caso médio:** $O(n \log n)$. Na prática as divisões continuam sendo mais ou menos equilibradas, mesmo não sendo perfeitas toda vez.
- **Pior caso:** $O(n^2)$. É quando o pivô escolhido é sempre o menor ou o maior elemento, aí a divisão fica bem desigual.
- **Vantagens:** É bem mais rápido que o Bubble Sort pra arrays grandes, e também organiza tudo dentro do próprio array.
- **Limitações:** Ele exige mais da memória, pois vai fazendo as divisões (usa a pilha de recursão). Também pode cair no pior caso dependendo de como o pivô é escolhido, e como ele troca elementos que estão distantes, pode acabar bagunçando a ordem original de elementos iguais.

- **Quando usar:** Em aplicações que têm grandes quantidades de dados, onde a velocidade importa mais.
- **Quando não usar:** Quando precisa garantir a ordem original de elementos iguais, ou quando precisa de um desempenho previsível mesmo no pior caso.

Característica	Bubble Sort	Quick Sort
Princípio de funcionamento	troca os elementos vizinhos que estão fora de ordem	escolhe um elemento (pivô) e divide o vetor entre os menores e os maiores que ele para fazer as comparações
Melhor caso	arrays praticamente já ordenados, com poucas trocas - $O(n)$	vetores que são divididos igualmente pelo pivô- $O(n \log n)$
Caso médio	arrays em ordem aleatória, com bastante troca ainda - $O(n^2)$	na prática as divisões continuam sendo mais ou menos equilibradas- $O(n \log n)$
Pior caso	arrays totalmente ao contrário (decrecente quando deveria ser crescente) - $O(n^2)$	divisão por apenas um elemento, quando o pivô é sempre o menor ou o maior- $O(n^2)$
Uso de memória	usa bem pouca memória, só uma variável pra fazer a troca	usa mais memória que o Bubble Sort por causa das chamadas recursivas
Vantagem principal	fácil de entender e usa menos memória	menor demora no tempo de execução
Limitação principal	em caso de grandes quantidades de dados, demora muito pra ser realizado	pode cair no pior caso dependendo de como o pivô é escolhido, e também não garante manter a ordem de elementos iguais
Aplicação recomendada	em aplicações pequenas com poucos números	grandes aplicações que utilizarão grande quantidade de dados

PARTE II

Tamanho do Array	Bubble Sort – Comparações	Bubble Sort – Trocas	Quick Sort – Comparações	Quick Sort – Movimentações
10	45	11	38	38
20	190	88	67	67
1.000	499.500	248.304	11.348	11.348

a) Qual algoritmo realizou menos operações para 10 elementos?

Se olharmos apenas as trocas foi o Bubble Sort, mas se juntar comparações + movimentações seria o Quick Sort.

b) O comportamento permaneceu igual para 20 elementos?

Não, o comportamento não permaneceu igual, houve uma inversão.

c) O que aconteceu quando o tamanho aumentou para 1.000 elementos?

A diferença se tornou ainda maior, com o Bubble fazendo muito comparações e trocas que o Quick.

d) Qual algoritmo apresentou maior crescimento da quantidade de operações?

O Bubble Sort

e) Os resultados experimentais são coerentes com as complexidades teóricas estudadas?

Sim, os resultados são coerentes com a teoria, o crescimento observado (~11.000x) bateu de forma muito próxima com o crescimento esperado pela fórmula $O(n^2)$

f) Em qual situação você escolheria Bubble Sort?

Em casos que possui poucos elementos para serem comparados e trocados.

g) Em qual situação você escolheria Quick Sort?

Em casos que possuem muitos elementos, onde a performance/velocidade é importante.

PARTE III

Matriz	Nº de elementos	Busca no início	Busca no final	Valor inexistente
2×2	4	1	4	4
10×10	100	1	100	100
100×100	10.000	1	10.000	10.000

a) Por que encontrar um elemento no início exige menos operações?

Porque a busca sequencial para assim que encontra o valor, então se ele está numa das primeiras posições percorridas, o algoritmo já retorna logo, sem precisar comparar o resto da matriz. Isso é confirmado na tabela: em todas as matrizes, a busca no início precisou de apenas 1 comparação.

b) O que acontece quando o elemento procurado não existe?

O algoritmo precisa percorrer a matriz inteira, comparando célula por célula, e só no final descobre que o valor não está lá — por isso é o caso que gera o maior número de comparações possível, igual ao número total de elementos da matriz.

c) Qual é o pior caso da busca sequencial?

É quando o valor procurado está na última posição da matriz, ou quando ele nem existe — nos dois casos o algoritmo precisa passar por todas as células antes de terminar, como mostrado na tabela (busca no final e inexistente sempre bateram no número total de elementos).

d) Como o aumento das dimensões da matriz influencia a quantidade de operações?

Quanto maior a matriz, mais comparações são necessárias na busca no final e na busca inexistente, já que o número de comparações cresce na mesma proporção do número total de células. Isso fica claro na tabela: 4, depois 100, depois 10.000 comparações, acompanhando exatamente o crescimento do tamanho da matriz.

e) Qual a complexidade da busca sequencial em uma matriz com m linhas e n colunas?

É $O(m \times n)$, pois no pior caso o algoritmo precisa percorrer todas as linhas e, dentro de cada linha, todas as colunas. Os resultados experimentais confirmam isso, já que o número de comparações no pior caso bateu exatamente com o total de elementos ($m \times n$) de cada matriz testada.

PARTE IV

Quantas operações de percurso foram necessárias:

Foram necessárias aproximadamente 40 operações de percurso, já que o programa realiza 4 loops separados, cada um percorrendo os 10 elementos do array: um loop para coletar as temperaturas, um para encontrar o maior valor, um para encontrar o menor valor, e um para contar/separar os valores acima da média.

Qual a complexidade do algoritmo desenvolvido:

A complexidade é $O(n)$, porque mesmo o programa tendo 4 loops, eles são sequenciais (um depois do outro), não aninhados (loop dentro de loop). Cada loop sozinho já é $O(n)$, e mesmo somando os 4 (o que daria algo como " $4n$ " operações), a notação Big O ignora essa constante multiplicativa, já que o que importa é a taxa de crescimento conforme o array aumenta, não o número exato de operações. Se o array de temperaturas tivesse 1000 elementos ao invés de 10, o número de operações cresceria de forma proporcional (linear), não exponencial.

Item	Valor
Temperaturas digitadas (índice 0 a 9)	19.5, 23.0, 17.8, 25.3, 21.1, 16.4, 28.7, 22.0, 18.9, 24.6
Média	21.73
Maior temperatura	28.7 (índice 6)
Menor temperatura	16.4 (índice 5)
Quantidade de valores acima da média	5
Valores acima da média	23.0, 25.3, 28.7, 22.0, 24.6
Operações de percurso (estimado)	~40
Complexidade do algoritmo	$O(n)$

PARTE V

Por que são necessários loops aninhados?

Porque com um loop só, seria possível acessar cada linha inteira (cada sensor com suas 24 medições), mas não seria possível acessar cada valor individual dentro dela para comparar, somar ou analisar. O segundo loop é necessário para "entrar" dentro de cada linha e percorrer cada posição individualmente.

Qual o papel dos índices `[i][j]`?

O índice `i` representa qual sensor (linha) está sendo consultado, e o índice `j` representa qual horário (coluna, de 0 a 23) daquele sensor. Juntos, `[i][j]` permitem identificar exatamente qual sensor registrou determinada medição e em que horário ela ocorreu.

Quantas posições da matriz são percorridas?

120 posições, já que são 5 sensores × 24 horários = 120 medições ao todo.

Qual a relação entre o número de linhas, colunas e quantidade de operações?

A quantidade de operações é igual ao número de linhas multiplicado pelo número de colunas (linhas × colunas). Essa relação corresponde à complexidade $O(m \times n)$ já identificada na Parte 3 — o número de operações cresce proporcionalmente ao produto das duas dimensões da matriz.

MATRIZ DE EXEMPLO

Horário	Sensor 1	Sensor 2	Sensor 3	Sensor 4	Sensor 5
0h	17.12	33.40	22.78	21.69	15.92
1h	29.12	34.96	34.01	23.71	26.79
2h	28.29	29.08	28.15	28.93	15.72
3h	16.72	29.45	32.26	15.27	22.49
4h	21.75	29.78	32.16	31.52	25.25
5h	24.52	28.00	31.09	22.59	17.59
6h	30.22	26.93	17.32	33.95	34.77
7h	15.32	32.41	21.32	15.82	17.68
8h	18.19	16.28	27.80	24.31	20.24

Horário	Sensor 1	Sensor 2	Sensor 3	Sensor 4	Sensor 5
0h	17.12	33.40	22.78	21.69	15.92
9h	28.40	19.99	23.08	32.75	29.50
10h	28.25	22.97	28.73	32.48	32.25
11h	24.41	20.91	16.21	22.15	16.93
12h	17.06	20.50	17.43	20.22	19.74
13h	16.16	15.94	15.42	24.00	17.45
14h	28.57	34.79	15.71	33.33	28.39
15h	27.21	18.13	22.58	22.72	27.40
16h	32.63	21.35	20.67	19.81	20.14
17h	21.16	30.59	26.45	27.29	23.01
18h	22.51	23.80	20.67	20.70	16.59
19h	17.99	25.01	18.37	17.88	31.30
20h	27.18	33.83	28.61	20.66	23.49
21h	25.19	24.15	21.27	20.71	29.61
22h	27.22	32.42	15.84	34.74	32.95
23h	27.22	25.80	18.29	19.13	33.82

MÉDIA DOS SENSORES

Sensor 1	Sensor 2	Sensor 3	Sensor 4	Sensor 5
23.66	26.27	23.18	24.43	24.13

Maior temperatura	Sensor responsável	Horário da ocorrência	Média geral (120 medições)	Limite informado	Qtd. leituras acima do limite
34.96	Sensor 2	1h	24.33	25 °C	53

PARTE VI – Análise e Conclusão

Ao longo dessa atividade, foram realizados experimentos comparando algoritmos de ordenação (Bubble Sort e Quick Sort) e de busca (busca sequencial em matrizes), além de exercícios práticos com arrays e matrizes (temperaturas e sensores). A seguir, uma análise comparando os resultados obtidos.

1. O aumento do tamanho da estrutura de dados influencia a quantidade de operações?

Sim. Em todos os experimentos realizados — tanto na ordenação (Bubble Sort e Quick Sort com 10, 20 e 1.000 elementos) quanto na busca em matrizes (2×2 , 10×10 e 100×100) — o aumento do tamanho da estrutura de dados sempre resultou em mais operações (comparações, trocas ou movimentações). Isso confirma que o tamanho da entrada é um fator direto no custo de processamento de qualquer algoritmo.

2. Bubble Sort e Quick Sort crescem da mesma maneira quando o número de elementos aumenta?

Não, eles não crescem da mesma forma. O Bubble Sort tem complexidade $O(n^2)$, enquanto o Quick Sort tem complexidade $O(n \log n)$. Isso ficou claro nos experimentos: quando o array cresceu 100 vezes (de 10 para 1.000 elementos), o Bubble Sort multiplicou suas operações por cerca de 11.000 vezes, enquanto o Quick Sort multiplicou por apenas cerca de 300 vezes. Isso mostra que o Bubble Sort piora muito mais rápido conforme os dados aumentam, enquanto o Quick Sort se mantém mais eficiente mesmo em arrays grandes.

3. Por que analisar somente o resultado final da ordenação não é suficiente para comparar algoritmos?

Os resultados finais são iguais — os dois algoritmos produzem o mesmo array ordenado. Mas o caminho e o tempo (número de operações) que cada um leva para chegar até esse resultado são bem diferentes. Se olharmos só o resultado final, não conseguimos perceber que o Bubble Sort precisou de muito mais comparações e trocas para ordenar a mesma quantidade de dados que o Quick Sort. Por isso, medir e comparar o número de operações (comparações, trocas, movimentações) é essencial para avaliar a eficiência real de um algoritmo — o resultado sozinho esconde o custo do processo.

Conclusão geral: os experimentos confirmam, na prática, o que a teoria de complexidade (Big O) prevê: o tamanho da entrada influencia diretamente o custo computacional, algoritmos diferentes crescem em ritmos diferentes conforme os dados aumentam, e apenas verificar se um algoritmo "funciona" (chega ao resultado certo) não é suficiente — é preciso medir seu desempenho para escolher a ferramenta certa para cada situação.